

Vulkan Demos - Shader & Rendering Pipeline Documentation

Generated Documentation - Read-only mapping of all shaders, rendering components, and Vulkan infrastructure.

Table of Contents

1. [Shader Mapping](#)
 2. [Vulkan Rendering Pipelines](#)
 3. [Render Targets \(Attachments\)](#)
 4. [Buffers](#)
 5. [Descriptor Sets & Bindings](#)
 6. [Rendering Flow](#)
-

Shader Mapping

1. GBuffer Shaders (Deferred Rendering)

`gbuffer.vert` - Vertex Shader

- **Purpose:** Transform vertex data and prepare per-fragment data for G-Buffer
- **Input Data Needed:**
 - Vertex positions, normals, UVs
 - Per-frame uniform buffer (projection, view, model matrices, camera position)
- **Data Being Pushed:** ✓ YES
 - UBO pushed every frame with camera matrices
- **Creation Status:** ✓ CREATED in `main.cpp` (line ~3008)
- **Descriptor Bindings:**
 - Set 1, Binding 0: UBO (projection, model, view, cameraPos)
- **Output:** Transformed positions, normals, UVs to fragment shader

`gbuffer.frag` - Fragment Shader

- **Purpose:** Write to G-Buffer attachments (Albedo, Normal+Specular)
 - **Input Data Needed:**
 - Interpolated position, normal, UV from vertex shader
 - UBO data (near/far planes)
 - **Data Being Pushed:** ✓ YES
 - UBO updated per frame with camera near/far planes
 - **Creation Status:** ✓ CREATED in `main.cpp`
 - **Descriptor Bindings:**
 - Set 1, Binding 0: UBO
 - **Output Attachments:**
 - Location 0: Normal+Specular (R16G16B16A16_SFLOAT) → `gBufferNormalSpecularRT`
 - Location 1: Albedo (R8G8B8A8_SNORM) → `gBufferAlbedoRT`
-

2. SSAO Shaders (Screen-Space Ambient Occlusion)

`fullscreen.vert` - Vertex Shader (Fullscreen Triangle)

- **Purpose:** Generate fullscreen triangle for post-processing passes

- **Input Data Needed:** None (generated via `gl_VertexIndex`)
- **Data Being Pushed:** × NO
- **Creation Status:** ✓ CREATED in `main.cpp`
- **Output:** Screen-space UVs (0,0 to 1,1)

`ssao.frag` - Fragment Shader

- **Purpose:** Compute screen-space ambient occlusion from depth and normals
- **Input Data Needed:**
 - G-Buffer depth texture (`depthRT`)
 - G-Buffer normal texture (`gBufferNormalSpecularRT`)
 - SSAO noise texture (random vectors)
 - SSAO kernel samples (32 random directions)
 - Inverse projection matrix (from UBO)
- **Data Being Pushed:** ✓ YES
 - SSAO Kernels: 32 * vec4 samples
 - SSAO Noise: Generated random texture
 - UBO: Projection matrix (inverse calculated in shader)
- **Creation Status:** ✓ CREATED in `main.cpp` (line ~3116)
- **Descriptor Bindings (Set 0):**
 - Binding 0: Depth RT (`texture2D`)
 - Binding 1: SSAO RT (`texture2D`)
 - Binding 2: SSAO Blur RT (`texture2D`)
 - Binding 3: GBuffer Albedo RT (`texture2D`)
 - Binding 4: GBuffer Normal Specular RT (`texture2D`)
 - Binding 5: Mesh textures array (unused here)
 - Binding 6: RT Sampler 1
 - Binding 7: RT Sampler 2
 - Binding 8: Texture sampler
 - Binding 9: SSAO Noise texture
 - Binding 10: SSAO Kernels UBO
- **Descriptor Bindings (Set 1):**
 - Binding 0: UBO (projection, model, view, cameraPos, near/far planes)
- **Output:** Single float occlusion factor → `ssaoRT`

`ssao_blur.comp` - Compute Shader

- **Purpose:** Blur SSAO results (currently empty stub)
- **Input Data Needed:** None implemented
- **Data Being Pushed:** × NO
- **Creation Status:** ✓ CREATED but NOT IMPLEMENTED (empty shader)
- **Current Status:** Placeholder for future blur pass

3. Composition Shader (Final Output)

`compose.frag` - Fragment Shader

- **Purpose:** Composite SSAO occlusion with albedo for final output
- **Input Data Needed:**
 - SSAO result texture (`ssaoRT`)
 - G-Buffer albedo texture (`gBufferAlbedoRT`)
- **Data Being Pushed:** ✓ YES
 - Textures pushed to descriptor set

- **Creation Status:** ✓ CREATED in `main.cpp` (line ~3262)
- **Descriptor Bindings:**
 - Set 0, Binding 1: SSAO RT
 - Set 0, Binding 3: GBuffer Albedo RT
 - Set 0, Binding 6: RT Sampler 1
- **Output:** Final composite color (albedo * ao)

4. Triangle Filtering Shader (Mesh Optimization)

`filtering.comp` - Compute Shader

- **Purpose:** Perform triangle culling/filtering using meshoptimizer techniques
- **Input Data Needed:**
 - Index buffer (array of triangle indices)
 - Vertex buffer (position data)
 - Vertex stride information
 - Index/vertex counts
- **Data Being Pushed:** ✓ YES
 - Push constants containing:
 - IndexBuffer (buffer reference)
 - VertexBuffer (buffer reference)
 - FilteredIndexBuffer[4] (output buffers)
 - vertexStride, indexCount, vertexCount
- **Creation Status:** ✓ CREATED in `main.cpp` (line ~2854)
- **Pipeline Type:** Compute (256 threads per group)
- **Output:** Filtered indices → `gTFilteredTriangles`

Vulkan Rendering Pipelines

Pipeline Summary Table

Pipeline Name	Type	Shaders	Status	Purpose
<code>gBufferPipeline</code>	Graphics	<code>gbuffer.vert</code> + <code>gbuffer.frag</code>	✓ Created	Deferred shading G-Buffer pass
<code>ssaoPipeline</code>	Graphics	<code>fullscreen.vert</code> + <code>ssao.frag</code>	✓ Created	SSAO computation
<code>ssaoBlurPipeline</code>	Graphics	<code>fullscreen.vert</code> + <code>ssao_blur.comp</code>	✓ Created	SSAO blur (not implemented)
<code>composePipeline</code>	Graphics	<code>fullscreen.vert</code> + <code>compose.frag</code>	✓ Created	Final composition
<code>gTFilteringPipeline</code>	Compute	<code>filtering.comp</code>	✓ Created	Triangle filtering/culling

Pipeline Details

`gBufferPipeline`

- **Location:** `src/demo/main.cpp`
- **Input Assembly:** Triangle list

- **Vertex Input:**
 - Binding 0: Vertex buffer (position, normal, UV)
- **Color Attachments:** 2
 - Attachment 0: gBufferNormalSpecularRT (R16G16B16A16_SFLOAT)
 - Attachment 1: gBufferAlbedoRT (R8G8B8A8_SNORM)
- **Depth Attachment:** gDepthBufferRT (D32_SFLOAT)
- **Dynamic State:** Viewport, scissor, depth compare, color write enable

ssaoPipeline

- **Location:** [src/demo/main.cpp](#)
- **Input Assembly:** Triangle list (fullscreen)
- **Vertex Input:** None (generated)
- **Color Attachments:** 1
 - Attachment 0: ssaoRT (R32_SFLOAT)
- **Depth:** Disabled
- **Dynamic State:** Viewport, scissor

composePipeline

- **Location:** [src/demo/main.cpp](#)
- **Input Assembly:** Triangle list (fullscreen)
- **Vertex Input:** None (generated)
- **Color Attachments:** 1
 - Attachment 0: Swapchain image (B8G8R8A8_SRGB)
- **Depth:** Disabled

gTFilteringPipeline (Compute)

- **Location:** [src/demo/main.cpp](#)
- **Local Group Size:** 256 x 1 x 1
- **Uses Buffer References:** Yes (EXT_buffer_reference extension)

Render Targets (Attachments)

Render Target Summary Table

Name	Format	Dimensions	Usage	Created	Start Layout
gBufferNormalSpecularRT	R16G16B16A16_SFLOAT	Window size	Color + Sampled	✔ Line 2194	SHADER_READ_ONLY
gBufferAlbedoRT	R8G8B8A8_SNORM	Window size	Color + Sampled	✔ Line 2178	SHADER_READ_ONLY
gDepthBufferRT	D32_SFLOAT	Window size	Depth + Sampled	✔ Line 2240	UNDEFINED
ssaoRT	R32_SFLOAT	Window size	Color + Sampled	✔ Line 2212	SHADER_READ_ONLY
ssaoBlurRT	R32_SFLOAT	Window size	Color + Sampled	✔ Line 2229	SHADER_READ_ONLY

Render Target Details

gBufferNormalSpecularRT

```

Format:      VK_FORMAT_R16G16B16A16_SFLOAT
Size:        gAppSettings->width x gAppSettings->height
Usage:       VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_SAMPLED_BIT
Clear Value: [0.0, 0.0, 0.0, 0.0]
Purpose:     Store encoded normals and specular values
Location:    main.cpp line 2194
Descriptor:  Set 0, Binding 4 (SSAO shader input)

```

- **Encoding:** Octahedral encoding of normal vectors (RG channels)
- **Specular:** BA channels (not used in current shaders)

gBufferAlbedoRT

```

Format:      VK_FORMAT_R8G8B8A8_SNORM
Size:        gAppSettings->width x gAppSettings->height
Usage:       VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_SAMPLED_BIT
Clear Value: [0.0, 0.0, 0.0, 0.0]
Purpose:     Store surface albedo/diffuse color
Location:    main.cpp line 2178
Descriptor:  Set 0, Binding 3 (SSAO and Compose shader inputs)

```

- **Current Content:** Hard-coded to green (0, 1, 0, 0) in gbuffer.frag
- **Future:** Should contain material albedo from textures

gDepthBufferRT

```

Format:      VK_FORMAT_D32_SFLOAT
Size:        gAppSettings->width x gAppSettings->height
Usage:       VK_IMAGE_USAGE_DEPTH_STENCIL_ATTACHMENT_BIT | VK_IMAGE_USAGE_SAMPLED_BIT
Clear Value: 1.0 (far plane)
Purpose:     Store depth for SSAO computation and depth testing
Location:    main.cpp line 2240
Descriptor:  Set 0, Binding 0 (SSAO shader input)

```

- **Aspect Mask:** VK_IMAGE_ASPECT_DEPTH_BIT
- **Attachment Type:** Depth stencil attachment

ssaoRT

```

Format:      VK_FORMAT_R32_SFLOAT
Size:        gAppSettings->width x gAppSettings->height
Usage:       VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_SAMPLED_BIT
Clear Value: [1.0, 1.0, 1.0, 1.0]
Purpose:     Store SSAO occlusion values (1.0 = fully lit, 0.0 = fully occluded)
Location:    main.cpp line 2212
Descriptor:  Set 0, Binding 1 (compose shader input)

```

- **Range:** [0.0, 1.0] where 1.0 means no occlusion

- **Output:** Single channel float (grayscale)

ssaoBlurRT

```
Format:      VK_FORMAT_R32_SFLOAT
Size:        gAppSettings->width x gAppSettings->height
Usage:       VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_SAMPLED_BIT
Clear Value: [1.0, 1.0, 1.0, 1.0]
Purpose:     Store blurred SSAO results (not yet implemented)
Location:    main.cpp line 2229
Descriptor:  Set 0, Binding 2
```

- **Current Status:** Created but not populated (ssao_blur.comp is empty)
- **Future:** Should contain spatially filtered SSAO

Buffers

Buffer Summary Table

Name	Size	Usage	Creation	Pushed Data	Purpose
uboBuffer[FRAMES_IN_FLIGHT]	sizeof(Uniform)	Uniform buffer	Per-frame	✓ Camera matrices	G-Buffer and SSAO shader data
ssaoKernelBuffer	32 * sizeof(vec4)	Uniform buffer	Init	✓ SSAO samples	SSAO kernel computation
gTFilteredTriangles	Variable	Storage buffer	Init	✗ Not pushed	Triangle filtering output
lightsDataBuffer[FRAMES_IN_FLIGHT]	Variable	Uniform buffer	Per-frame	✗ Not used	Future lighting data
shadowGenBuffer[FRAMES_IN_FLIGHT]	Variable	Uniform buffer	Per-frame	✗ Not used	Future shadow generation
Swapchain buffers	Window size	Color	System	✓ Composite output	Final screen output

Buffer Details

uboBuffer (Uniform Buffer Object)

```
Frames in Flight: 2
Per-Frame Size:  sizeof(Uniform)
Members:
- proj:    mat4 (projection matrix)
- model:   mat4 (model transformation)
- view:    mat4 (camera view matrix)
```

```

- cameraPos: vec3 (camera position)
- nearPlane: float
- farPlane: float
Location:    main.cpp ~line 1317
Descriptor:  Set 1, Binding 0
Update:      Every frame with current camera matrices
Used By:     gbuffer.vert, gbuffer.frag, ssao.frag

```

Data Being Pushed: ✓ YES

- Updated every frame with:
 - Camera position
 - View matrix (from camera.hpp)
 - Projection matrix
 - Near/far planes for depth reconstruction

ssaoKernelBuffer

```

Size:        32 * sizeof(glm::vec4)
Members:     32 random kernel samples in hemisphere
Location:    main.cpp line 1317
Descriptor:  Set 0, Binding 10
Generation:  generateSSAOKernel(SSAO_KERNEL_SIZE, true)
Update:      Once at initialization (deterministic seed)
Used By:     ssao.frag

```

Data Being Pushed: ✓ YES (at init)

- 32 random directions in hemisphere
- Deterministic seeding for reproducibility
- Used for ambient occlusion sampling

ssaoNoise Texture (Generated)

```

Size:        4 x 4 pixels
Format:      Texture
Members:     Random XY directions (Z=0)
Location:    main.cpp ~line 1307
Descriptor:  Set 0, Binding 9
Generation:  generateSSAONoise(SSAO_NOISE_DIM)
Update:      Once at initialization
Used By:     ssao.frag (tile-able rotation noise)

```

gTFilteredTriangles

```

Type:        Storage Buffer
Purpose:     Output of triangle filtering compute shader
Status:      ✓ Created
Data Pushed: ✗ NO (computed by shader)
Used By:     Future rendering passes
Location:    main.cpp line 308

```

Descriptor Sets & Bindings

Descriptor Set Layout Architecture

The application uses **2 descriptor set layouts**:

Layout 0: Persistent Set (Set 0)

- **Persistence**: Constant across frames
- **Binding Count**: 11 bindings
- **Update Frequency**: Initialization only

Binding	Type	Resource	Format	Purpose
0	Sampled Image	gDepthBufferRT	D32_SFLOAT	Depth input to SSAO
1	Sampled Image	ssaoRT	R32_SFLOAT	SSAO result
2	Sampled Image	ssaoBlurRT	R32_SFLOAT	Blurred SSAO (unused)
3	Sampled Image	gBufferAlbedoRT	R8G8B8A8_SNORM	Albedo input
4	Sampled Image	gBufferNormalSpecularRT	R16G16B16A16_SFLOAT	Normal input
5	Sampled Image	meshTextures[2]	Variable	Material textures
6	Sampler	rtSampler1	Linear	RT sampling
7	Sampler	rtSampler2	Linear	RT sampling
8	Sampler	textureSampler	Linear	Texture sampling
9	Sampled Image	ssaoNoise	R32G32B32A32	SSAO rotation noise
10	Uniform Buffer	ssaoKernelBuffer	-	SSAO kernel samples

Creation Location: main.cpp line 1699-1850

Layout 1: Per-Frame Set (Set 1)

- **Persistence**: Changes per frame
- **Binding Count**: 2 bindings
- **Update Frequency**: Every frame
- **Instances**: 2 (for double buffering)

Binding	Type	Resource	Format	Purpose
0	Uniform Buffer	uboBuffer[frame]	-	Camera matrices & transforms

Creation Location: main.cpp line 1750+

Rendering Flow

Frame Rendering Sequence




```

├─ Pipeline: gBufferPipeline
├─ Vertex Shader: gbuffer.vert
│   └─ Input: Vertex data (position, normal, UV)
│   └─ UBO: Model, View, Projection matrices
├─ Fragment Shader: gbuffer.frag
│   └─ Output[0]: Normal+Specular → gBufferNormalSpecularRT
│   └─ Output[1]: Albedo → gBufferAlbedoRT
│   └─ Depth: → gDepthBufferRT
└─ Result: Geometric data in G-Buffer

├─ 2. IMAGE LAYOUT TRANSITIONS
│   └─ gBufferNormalSpecularRT: ATTACHMENT → SHADER
│   └─ gBufferAlbedoRT: ATTACHMENT → SHADER
│   └─ gDepthBufferRT: ATTACHMENT → SHADER
├─ 3. SSAO PASS
│   └─ Pipeline: ssaoPipeline
│   └─ Vertex Shader: fullscreen.vert
│       └─ Generate fullscreen triangle
│   └─ Fragment Shader: ssao.frag
│       └─ Input: gDepthBufferRT, gBufferNormalSpecularRT
│       └─ Uniform: ssaoKernelBuffer (32 samples)
│       └─ Texture: ssaoNoise (rotation noise)
│       └─ Output: Occlusion factor → ssaoRT
│   └─ Result: Screen-space ambient occlusion
├─ 4. IMAGE LAYOUT TRANSITIONS
│   └─ ssaoRT: ATTACHMENT → SHADER
├─ 5. COMPOSITION PASS
│   └─ Pipeline: composePipeline
│   └─ Vertex Shader: fullscreen.vert
│   └─ Fragment Shader: compose.frag
│       └─ Input: gBufferAlbedoRT, ssaoRT
│       └─ Formula:  $outColor = albedo * ao$ 
│       └─ Output: Final composite → Swapchain
│   └─ Result: Final image ready for presentation
├─ 6. PRESENT
│   └─ Swapchain image presented to screen
└─ END FRAME

```

Per-Frame Data Updates

Every Frame:

1. Update `uboBuffer[currentFrame]` with:
 - Current camera position
 - Updated view matrix
 - Updated projection matrix
 - Near/far planes

At Initialization:

1. Generate SSAO kernel samples (32 random directions)
2. Generate SSAO noise texture (4x4 random vectors)
3. Create all render targets

4. Create all pipelines
5. Allocate descriptor sets

Component Creation Status Checklist

✓ FULLY CREATED & IMPLEMENTED

- ☒ GBuffer Pipeline (graphics)
- ☒ GBuffer Vertex Shader (gbuffer.vert)
- ☒ GBuffer Fragment Shader (gbuffer.frag)
- ☒ SSAO Pipeline (graphics)
- ☒ SSAO Fragment Shader (ssao.frag)
- ☒ Compose Pipeline (graphics)
- ☒ Compose Fragment Shader (compose.frag)
- ☒ Triangle Filtering Pipeline (compute)
- ☒ Triangle Filtering Shader (filtering.comp)
- ☒ Fullscreen Vertex Shader (fullscreen.vert)
- ☒ GBuffer Normal Specular Render Target
- ☒ GBuffer Albedo Render Target
- ☒ Depth Render Target
- ☒ SSAO Render Target
- ☒ SSAO Blur Render Target
- ☒ UBO Buffers (per-frame)
- ☒ SSAO Kernel Buffer
- ☒ SSAO Noise Texture
- ☒ Descriptor Sets (persistent & per-frame)
- ☒ Samplers (linear)
- ☒ Pipeline Layout with descriptor sets

⚠ CREATED BUT NOT FULLY IMPLEMENTED

- ☐ SSAO Blur Shader (ssao_blur.comp) - Empty stub, not computing blur
- ☐ Triangle Filtering Output - Computed but not used in rendering pipeline

✗ NOT YET CREATED

- ☐ Shadow mapping pass
- ☐ Lighting pass
- ☐ Material texture loading
- ☐ Dynamic mesh updates via filtering

Key Implementation Notes

1. **Double Buffering:** UBO and per-frame sets use `FRAMES_IN_FLIGHT` (2) to prevent GPU stalls
2. **Dynamic Rendering:** Using `VK_KHR_DYNAMIC_RENDERING` extension (no traditional render passes)
3. **Buffer References:** Triangle filtering uses `GL_EXT_buffer_reference` for flexible mesh access
4. **Octahedral Encoding:** Normals encoded as 2-channel values for efficient storage
5. **SSAO Deterministic:** Kernel generation uses deterministic seeding for reproducibility
6. **Fullscreen Triangle:** Used for post-processing via `gl_VertexIndex`
7. **Image Layout Management:** Careful transitions between `ATTACHMENT` and `SHADER_READ` layouts

References

- **Main Implementation:** [src/demo/main.cpp](#)
- **Shader Directory:** [shaders/](#)
- **Camera System:** [foundation/camera.hpp](#)
- **Build System:** [CMakeLists.txt](#)